# Final Project:

# What's the Number? A CNN Accelerator Simulator in C++

**Due Date: 2026/01/03 (Sat.) 23:59:59**

2025/11/25 ver2

2026/01/01 ver3

## 1. Background

In recent years, the rapid advancement of deep learning, particularly in fields such as computer vision and natural language processing, Convolutional Neural Networks (CNNs) have become the core engine of modern AI computation. However, the computational demands of these complex models present a critical bottleneck. Specifically, executing CNN models necessitates massive computation and, crucially, frequent data movement. This results in significant challenges for deployment, including high power consumption and low efficiency.

To meet these computational demands efficiently, researchers have increasingly turned to hardware acceleration. Recent research into AI hardware accelerators highlights one of the biggest bottlenecks: memory access. Studies have consistently shown that memory access consumes significantly more energy than the multiply-accumulate (MAC) operations, which is the fundamental arithmetic of deep learning themselves.

Therefore, modern AI accelerator designs aim to "compute more while moving less data." This leads to specialized architecture where dataflow and processing elements (PE) are carefully optimized.[1]

This project focuses on simulating and validating such a CNN hardware module, allowing us to explore how to minimize memory access and maximize computational throughput through dataflow design and PE array mapping.

## 2. Prerequisite Knowledge

**#1 The C++ Advantage: Why Not Python?**

You might wonder why we aren't using Python to finish the simulation since Python is typically used to design and train the CNN model itself. Here are a couple of essential reasons:

a. Precision and Fidelity: C++'s deterministic execution allows for the accurate modeling of low-level operations and timing.

b. Performance and Efficiency: Being a compiled language, C++ provides superior speed and fine-grained control over memory and hardware resources, which is vital for studying power and data movement bottlenecks.

While Python is excellent for high-level model definition, C++ is necessary to achieve the speed and accuracy required for a hardware accelerator simulator.

```
PS C:\Users\Jing\Desktop\114-Final-Project> .\mnist.exe
Predict: 5
accumulate time = 626000 ns (0.626 ms)
C++ execution time: 0.008162 seconds
PS C:\Users\Jing\Desktop\114-Final-Project> python python_version.py
Predict: 5
Accumulate time = 626000 ns (0.626 ms)
Python execution time: 0.012900 seconds
```

Figure 1 Speed Comparison Between Python and C++

**#2 From Algorithm to Silicon: Simulating Hardware Acceleration in C++**

The general design flow of a hardware accelerator typically includes algorithm design (Algorithm Level), architecture design (Architecture Level), hardware description (RTL / Verilog), and verification and simulation (Simulation & Verification).

In the early stage of architecture design, engineers usually develop a behavioral simulator to validate the design concept.

Through simulation, we can verify computational correctness, analyze data movement counts and computational efficiency, and explore how different dataflows affect overall performance.[2]

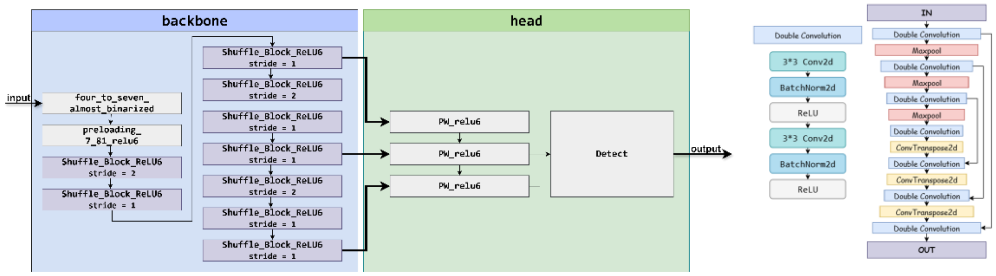



Figure 2 CNN Architecture Diagram Examples

This Final Project focuses on the architecture-level simulation stage, implementing the operational logic of a CNN accelerator using C++.

Meanwhile, to further enhance performance and reduce power consumption, we introduce a quantization mechanism that makes the data format more hardware friendly.

So, what exactly is **quantization**?

**#3 Quantization Matters: From FP32 to FP8, Shrinking Bits, Saving Power**

Quantization is essentially the process of compressing data representation to reduce the number of bits used. Although this process leads to a loss of numerical precision, it significantly reduces memory requirements and lowers power consumption. You can think of quantization as compressing a high-resolution image into a lower-quality version: the details decrease, but the file becomes smaller and faster to process, which is a trade-off that is particularly valuable in hardware systems.

To understand quantization, we must first recall how numerical data is

represented in computers using the standard FP32 (32-bit floating point) format, which is the typical representation for the float type in programming. FP32, as its name implies, consists of 32 bits, divided as follows: 1 sign bit + 8 exponent bits + 23 mantissa bits. To quantize the data, we aim to store data using a more compact format, FP8, which consists of 8 bits only: 1 sign bit + 4 exponent bits + 3 mantissa bits. [3] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, R. Ho, M. Jones, N. Jouppi, J. Kurzak, N. Luehr, S. Markidis, M. Mudigonda, J. Park, S. Pennycook, M. Siu, and C. Sun, "FP8 formats for deep learning," *arXiv preprint* arXiv:2209.05433, 2022.

[4]




Figure 3 Structure of FP32 and FP8

FP8 through the following steps:

- Decompose the FP32 value into its **sign**, **exponent**, and **mantissa** components.
- **Compress** the exponent and mantissa fields to fit into 8 bits.
- **Store** the result as an 8-bit integer representation.
- **Decode** it back to FP32 to evaluate the quantization error.

The **quantization error** is measured using the **Relative Error (%)**, which indicates how much information loss occurs during quantization.

$$ErrorRate = \frac{original\ -\ quantized}{original} \times 100\%$$

$$ErrorRate_{avg.} = \frac{\sum_{i=0}^{n} ErrorRate_i}{n}$$

| Binary | Hex | Value | Notes |
|---|---|---|---|
| 0 0000 000 | 0 | 0 | |
| 0 0000 001 | 1 | $2^{-6} \times (0 + \frac{1}{8}) = 0.001953125$ | Smallest positive subnormal number |
| 0 0000 111 | 7 | $2^{-6} \times (0 + \frac{7}{8}) = 0.013671875$ | Largest subnormal number |
| 0 0001 000 | 8 | $2^{-6} \times (1 + \frac{0}{8}) = 0.015625$ | Smallest positive normal number |
| 0 0101 011 | 2b | $2^{-2} \times (1 + \frac{3}{8}) = 0.34375$ | Nearest value to 1/3 |
| 0 0110 111 | 37 | $2^{-1} \times (1 + \frac{7}{8}) = 0.9375$ | Largest number less than one |
| 0 0111 000 | 38 | $2^{0} \times (1 + \frac{0}{8}) = 1$ | One |
| 0 0111 001 | 39 | $2^{0} \times (1 + \frac{1}{8}) = 1.125$ | Smallest number larger than one |
| 0 0111 100 | 3c | $2^{0} \times (1 + \frac{3}{8}) = 1.375$ | Nearest value to sqrt(2) = 1.414 |
| 0 1000 101 | 45 | $2^{1} \times (1 + \frac{5}{8}) = 3.25$ | Nearest value to PI = 3.141 |
| 0 1110 111 | 77 | $2^{7} \times (1 + \frac{7}{8}) = 240$ | Largest normal number without range extension |
| 0 1111 110 | 7e | $2^{8} \times (1 + \frac{6}{8}) = 448$ | Largest normal number |
| 0 1111 111 | 7f | $NaN$ | First of 2 NaN representations |
| 1 0000 000 | 80 | 0 | |
| 1 1000 000 | c0 | $-2$ | |
| 1 1111 111 | ff | $NaN$ | Last of 2 NaN representations |

Figure 4 Data Representation in the FP8 Floating-Point Format

## #4 What Does CNN Compute?

A Convolutional Neural Network (CNN) is a type of deep learning model inspired by the human visual system. It automatically learns to extract important features from images, such as edges, shapes, and objects, and is widely used in image classification, face recognition, and object detection.

CNN computation mainly consists of the following operations:

- Convolution: Uses small filters (kernels) sliding over the image to extract spatial features like edges or textures.
- Activation Function: Adds non-linearity to the model. A common example is ReLU (Rectified Linear Unit), which sets negative values to 0.
- Pooling: Reduces the size of feature maps and the overall computation cost. Common methods include Max Pooling and Average Pooling.
- Fully Connected Layer: Integrates the extracted features for the final classification or prediction.

Overall, CNN computation can be viewed as a series of data transformations: The input images will undergo feature extraction through convolution, be subsequently fed into fully connected layers for classification, and ultimately yield the prediction. In this project, we will focus on Convolution Layers and Fully Connected Layers, as these are the two computation types implemented in the simulator. [5]

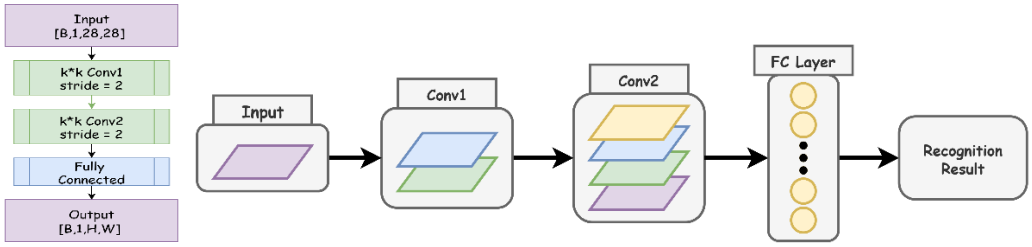



Figure 5 CNN Architecture Diagram of this Final Project

## #5 Convolution in Action: The Heartbeat of CNN Computation

The Convolutional Layer performs multiply-and-accumulate (MAC) operations on

input images or feature maps to extract meaningful features. Its core components include:

- Kernel / Filter: A small matrix (e.g., 3×3) used to detect local patterns.
- Stride: The step size when sliding the kernel.
- Padding: The border-handling method that controls output size

$$\text{output}(i,j) = \sum(\text{input} * \text{weight}) + \text{bias}$$

In essence, the convolution layer functions as a feature detector. The kernel slides over the input image, and at each position, it computes the dot product between the kernel and the corresponding image region.

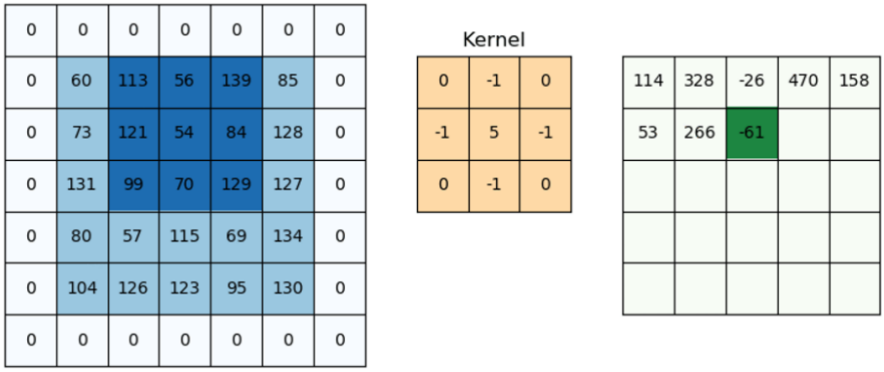



Figure 6 Convolution Computation

For example, assume an input of 7×7, a 3×3 kernel, padding = 1, and stride = 1. The resulting output feature map will be 5×5, requiring: $5 \times 5 \times 3 \times 3 = 225$ MAC operations. You can click on the image to see the animated calculation simulation.[5]

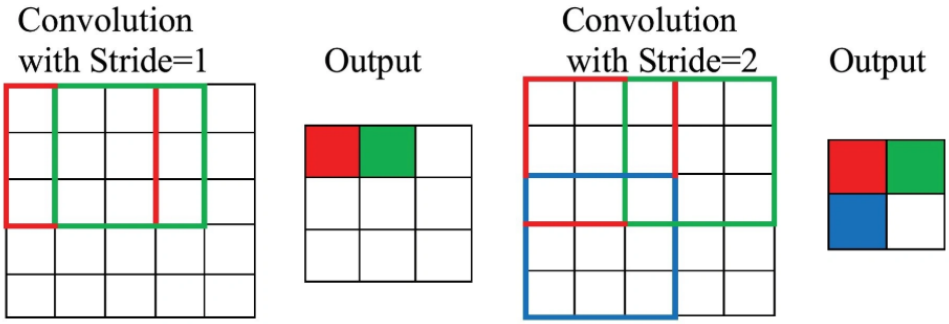



Figure 7 Convolution Computation with Different Strides

As input size increases, the number of computations grows rapidly. This is why CNNs demand massive computation and frequent data movement.

In this project, padding is ignored, and stride is set to 2, to make the computation easier and more efficient.

When the convolution layer has multiple input channels, each output feature map is produced by multiple 3×3 kernels, one per input channel. Each kernel performs convolution with its corresponding input channel, and the results are summed across all

channels to form a single output feature map. For example, if the input has 2 channels and the kernel size is 3×3, each output feature map requires 2 × (3×3) weights, and the total computation becomes 72 MAC operations.

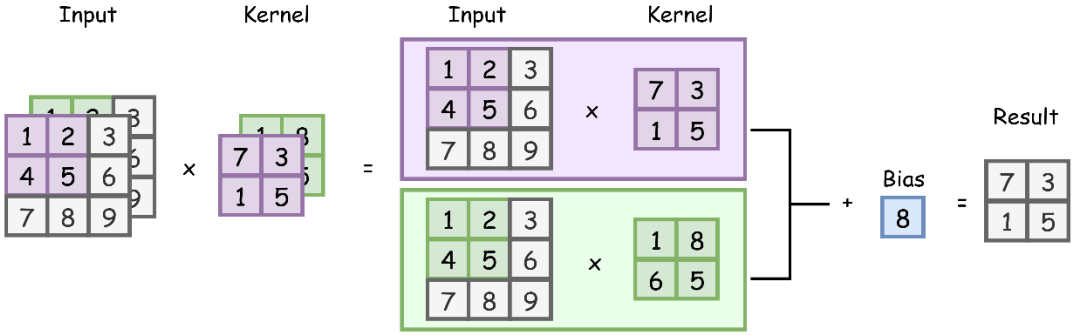



Figure 8 Multiple Input Channels Convolution Computation

**#6 Fully Connected Layers: Where Features Meet Decisions**

The Fully Connected Layer (FC Layer) maps the features extracted by convolution layers into the final classification output. Each output neuron connects to all input neurons, performing a matrix multiplication followed by accumulation:

$$y = Wx + b, W = \text{weight}, B = \text{Bias}$$

For example, if we have 9 input data points and want to classify them into 10 categories, we will obtain 10 sets of filters, each containing 9 weights and 1 bias. The highest output is 58.64 at digit 5, so the predicted handwritten number is 5.

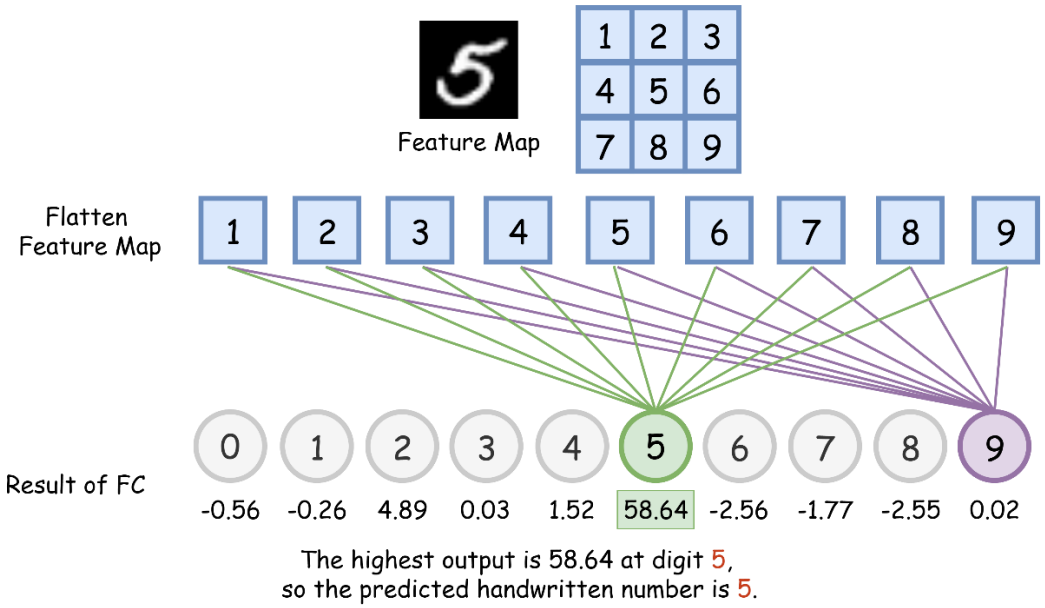



Figure 9 Fully Connected Computation

However, FC layers involve intensive computation and frequent memory access, often becoming the performance bottleneck of CNN models. Since the FC layer typically appears only once at the end of a CNN, hardware accelerators rarely optimize it. Instead, this layer is usually handled by the CPU.

In this project, you will design and simulate the matrix multiplication and accumulation

process to complete the forward pass of the CNN model.

**#7 Inside the Engine Room: Understanding the PE Array**

Finally, let's introduce the core computational engine of an AI hardware accelerator, which is the PE (Processing Element) Array.

A Processing Element (PE) is the smallest compute unit in the accelerator, responsible for MAC (Multiply-and-Accumulate) operations. A PE Array, as its name suggests, consists of multiple PEs arranged in a grid to support highly parallel computation.

In this project, the PE Array adopts a Weight-Stationary (WS) dataflow strategy. This means that weights remain fixed within each PE, while input activations flow across different PEs for computation. The main advantage is the reduction of redundant weight fetches, leading to lower memory access and power consumption.

Data movement within the PE Array operates as follows:

- Activations propagate horizontally across the array.
- Partial sums (PSum) are accumulated vertically between PEs.

Through this architecture, the PE Array effectively simulates the parallel behavior of CNN computations, improving both data reuse and computational efficiency.[6]

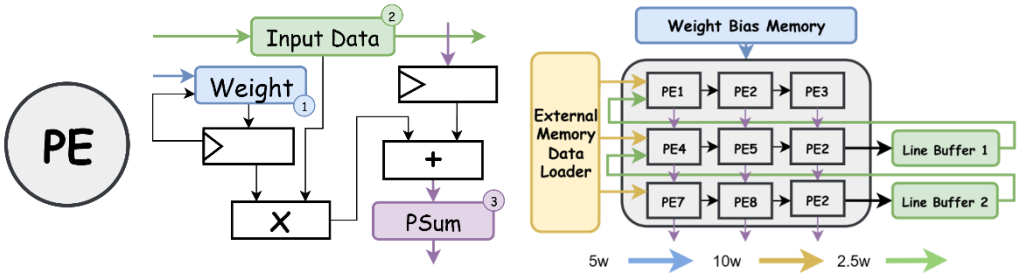



Figure 10 Architecture of PE Array

# 3. Problem Description

This semester's final project requires implementing a behavioral-level simulator of an AI hardware accelerator using C++. The implementation should simulate the mapping and dataflow of CNN kernels on a PE (Processing Element) Array, and perform forward inference on the MNIST dataset (28×28 handwritten digits).You must complete the entire process, from data input, quantization, convolution operations, to the fully connected layer. Also, evaluate both memory access energy consumption and the number of execution steps of the accelerator. The architecture diagram of the assignment is shown below:

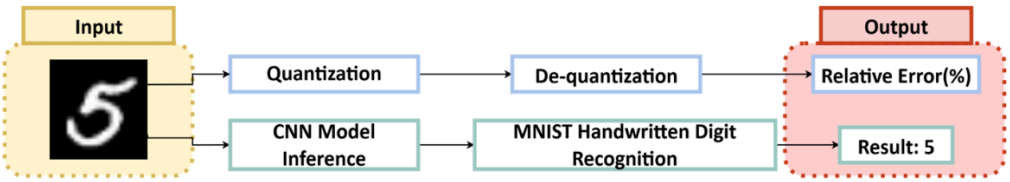



Figure 11 Architecture of Final Project

Your task is to read an input image from a txt file, first convert the image data to FP8 format and compute the quantization error, then use the simulator to perform MNIST handwritten digit inference **on the quantized data**, and finally write the prediction results and evaluation metrics to execution_result.txt. Overall performance will be evaluated by total energy consumption and the number of execution step, with smaller values being better. Below, each of the two parts will be explained in detail.

**Part 1: Quantization**

The objective of this part is to understand how quantization changes data formats and affects numerical precision, as well as to observe the errors introduced when converting from FP32 to FP8. You are required to read the provided input image file (a 28×28 floating-point txt file), perform quantization on each pixel value, convert the data from FP32 to FP8, and calculate the precision loss caused by quantization.

Through this process, you will be able to analyze the differences in data representation across different precision formats and understand how bit-width reduction impacts data accuracy and hardware power consumption. For detailed explanations of data formats and error analysis methods, please refer to the section "**#3 Quantization Matters: From FP32 to FP8, Shrinking Bits, Saving Power**". In our quantization setup, support for subnormal numbers has been disabled. Consequently, any values below the smallest normal number (2^{-6} = 0.015625) cannot be represented and are instead flushed to zero. Please perform all subsequent calculations based on this logic.

**Part 2: CNN computation**

The CNN computation in this part is designed to be relatively simple. The input data consists of 28×28 handwritten digit images and is processed through two convolutional layers. The first convolutional layer transforms the single-channel input image into two feature maps, and the second convolutional layer further converts these two feature maps into four feature maps.

After the two convolutional layers, all feature maps are flattened and fed into a fully connected layer, producing a 10-dimensional output corresponding to the classification of digits 0–9.

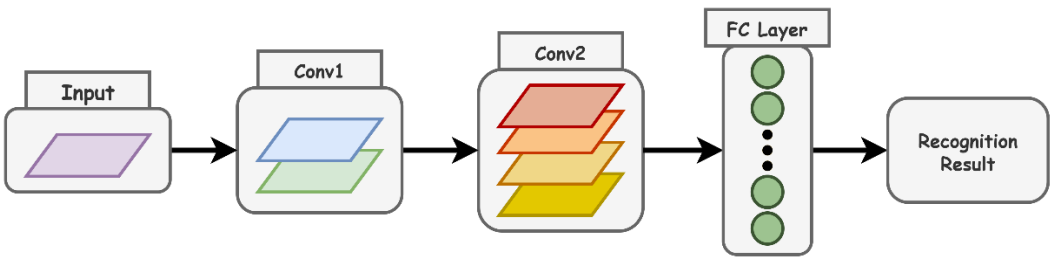



Figure 12 CNN Architecture Diagram of this Final Project

The size of the feature maps at each layer is determined by the kernel size, stride, and padding. In this final project, padding is not considered, and stride is fixed at 2.

- **Convolutional Layers:**

For this part of the computation, you are required to use the specified accelerator module to perform the convolution operations and are not allowed to replace or design their own hardware modules. This requirement is intended to verify the performance and adaptability of the accelerator. The main hardware architecture and operational details of the simulator are defined in the corresponding <hardware_architecture.h> header, and the following section will explain the operational flow starting from the architecture of a single PE (Processing Element) on the left.

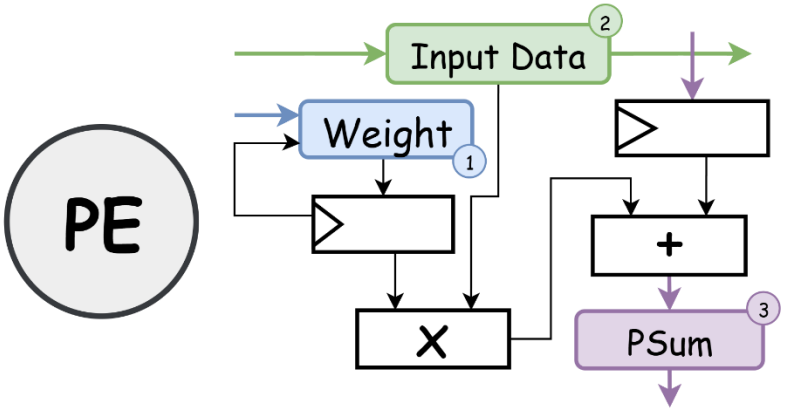
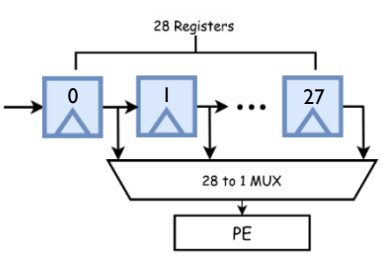



Figure 13 Architecture of PE Array

First, this hardware design involves three main types of data: **weight**, **input data**, and **PSum (partial sum)**. As mentioned earlier, the hardware adopts a **weight-stationary** strategy, meaning that weights are stored in the PE until new weights need to be loaded.

Next is the **input data**, which can come from either the **data loader** or the **line buffer**. After each computation, the input data is transmitted horizontally to the right. The **data loader** reads data from external memory and can be directly assigned, but accessing off-chip memory consumes more energy. The **line buffer** is a large register array composed of 28 connected registers. In this project, we represent it using a float array, and you can read data by selecting any specific element.

Finally, **PSum** represents the temporary result computed by the previous PE. After each computation, it is transmitted vertically downward. It corresponds to the accumulated result of each multiply-accumulate operation and can be read for subsequent computations.




```cpp
// Get item from line buffer
float line_buffer_get_item(int item, int buffer_number){
    if (buffer_number == 1) {
        return line_buffer_array_1[item];
    }
    else {
        return line_buffer_array_2[item];
    }
}
```

Figure 14 Architecture of Line Buffer and C++ Code

Next, we explain how data is transferred between memory units. Reading data from storage consumes energy, with the following rules: updating a **weight** costs **5w**; reading and inputting an **input data/activation** from the **External Memory Data Loader** costs **10w**; and reading and inputting an **input data/activation** from the **line buffer** costs **2.5w**.

You do not need to manually calculate the energy consumption, as the <hardware_architecture.cpp> module will handle it automatically. However, the results must be saved to execution_result.txt. Since accessing data from external memory consumes the most energy, designing an efficient data reuse strategy can greatly reduce overall energy consumption. Note that both the total number of steps and total energy consumption should represent the combined results of the two convolution operations.

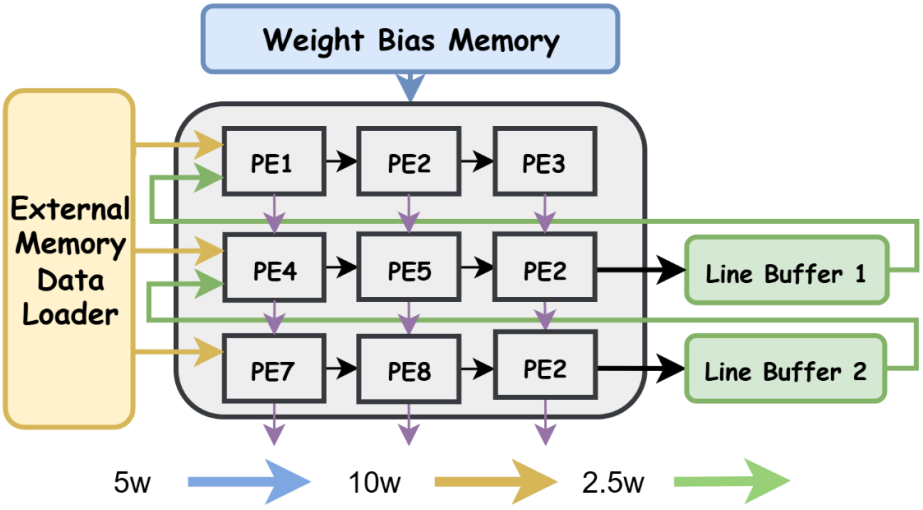



Figure 15 PE Array Dataflow and Memory Access Architecture

You now have a PE array consisting of a 3×3 processing element (PE) grid with two line buffers. Data can only be fed into the array from the leftmost PEs (PE1, PE4, PE7). With each computation step, the partial sum (PSum) propagates downward by one row, while the input data shifts right by one column.

The challenge is to design a dataflow strategy that determines how input data is supplied. It can either come directly from the data loader or be retrieved from the line buffers, so that the energy consumed by memory accesses and the number of PE computations are minimized.

Input kernels may vary in size (3×3, 5×5, 7×7) but are fixed within one test case.

By decomposing larger kernels into smaller blocks compatible with the 3×3 PE array and the hardware characteristics, an efficient algorithm can be designed for feeding input data, maximizing data reuse and minimizing energy consumption.

- **Fully Connected Layer:**

Based on "**#6 Fully Connected Layers: Where Features Meet Decisions**", the computation method of the fully connected layer can be understood. For this part, you do not need to use the header files specified in the assignment; you are free to design the implementation (but you must not use any libraries prohibited in the assignment restriction list—please refer to the assignment constraints). Finally, store the prediction results in result.txt.

# 4. Limitation

- Only the following libraries are permitted for this project. Do not use any others, or you will get 0 for your final project: <iostream>, <fstream>, <string>, <vector>, <cmath>, <iomanip>
- When computing the convolution layers, you are required to use the provided accelerator module in < hardware_architecture.h >. Do not replace it with your own hardware module.
- You cannot print the sample output directly as the answers for the test cases.

# 5. Input

The test case files will be provided for you to perform the calculations. The way the weights and biases are stored in each file is shown below.

| | |
|---|---|
| img_label.txt | Contains the handwritten digit image data, consisting of 28×28 data points represented as floating-point values. |
| conv1_weight.txt | Contains two sets of n×n weight data, where each row includes *n* values, and every *n* rows form one complete set. (The arrangement of the weights within the file will also be explained.) |
| conv1_bias.txt | Contains two bias values corresponding to the two output channels. |
| conv2_weight.txt | Contains eight sets of n×n weight data. Each row includes n values, and every n rows form one complete set. Every two consecutive sets correspond to the kernels for one output channel. |
| conv2_bias.txt | Contains four bias values corresponding to the four output channels. |
| fc_weight.txt | Suppose the output of conv2 (after flattening) contains m data points, then this file includes m×10 data points, where each group of m values corresponds to one output class. |
| fc_bias.txt | Contains the bias values for the ten output classes. |

| | |
|---|---|
| FOR OUTPUT CHANNEL 1 | 0.050215,-0.043871,0.038356<br>-0.019326,0.003426,0.025519<br>0.034402,-0.035647,0.070371 |
| FOR OUTPUT CHANNEL 2 | -0.034575,-0.090211,-0.112682<br>-0.037799,-0.141324,-0.186931<br>-0.040603,-0.094412,-0.091309 |

Figure 16 Data Distribution in conv1_weight.txt

| -0.032718,0.123313 | |
|---|---|
| FOR OUTPUT CHANNEL 1 | FOR OUTPUT CHANNEL 2 |

Figure 17 Data Distribution in conv1_bias.txt

| | | |
|---|---|---|
| FOR OUTPUT CHANNEL 1 | FOR INPUT CHANNEL 1 | 0.109593,0.085738,0.112907<br>-0.001812,-0.420564,0.258541<br>0.149408,-0.082909,0.066076 |
| | FOR INPUT CHANNEL 2 | 0.016544,0.142134,-0.049385<br>-0.057073,-1.361409,0.066671<br>0.234157,-0.187411,0.390726 |
| FOR OUTPUT CHANNEL 2 | FOR INPUT CHANNEL 1 | -0.121745,-0.056585,0.199143<br>-0.081433,-0.050261,0.051866<br>-0.032044,-0.095212,0.084737 |
| | FOR INPUT CHANNEL 2 | 0.369426,-0.557695,-0.391173<br>0.558640,0.123567,-0.426986<br>-0.296421,0.095806,-0.000164 |
| FOR OUTPUT CHANNEL 3 | FOR INPUT CHANNEL 1 | -0.193732,-0.079997,-0.167232<br>-0.143264,-0.024650,0.187036<br>-0.014838,-0.155067,-0.110293 |
| | FOR INPUT CHANNEL 2 | -0.052840,-0.014718,0.075239<br>-0.217076,-0.272656,-0.791490<br>0.411246,1.046007,-0.127751 |
| FOR OUTPUT CHANNEL 4 | FOR INPUT CHANNEL 1 | -0.034029,-0.103626,0.062166<br>0.016373,0.065933,0.050024<br>0.023864,-0.074234,-0.305428 |
| | FOR INPUT CHANNEL 2 | -0.047826,-0.550968,-0.959365<br>-0.000778,0.283835,0.826154<br>0.330821,0.366947,0.401556 |

Figure 18 Data Distribution in conv2_weight.txt

# 6. Output

| | |
|---|---|
| result_of_conv1.txt | The output feature map is of size N×N, with N values per row and a total of N rows. Each value in a row is separated by a comma, and the last value in each row is followed by a newline (without a trailing comma). All values are displayed up to three decimal places, truncated without rounding. This file is used for code correctness verification. |
| result_of_conv2.txt | The format is identical to result_of_conv1.txt. This file is also used for code correctness verification. |
| result_of_fc.txt | Contains a total of 10 values separated by commas, each |

| | |
|---|---|
| | displayed up to three decimal places, truncated without rounding. This file is used for code correctness verification. |
| execution_result.txt | Contains the quantized precision difference (average relative error (%), round to two decimal places), the prediction result, total energy consumption(displayed up to one decimal place), and execution step count. The relative error and result of the prediction will be used to verify the correctness of your code, while energy and step count will be used to evaluate memory and data reuse optimization performance. |




Figure 19 Data Distribution in result_of_fc.txt

# 7. Samples

After compiling and executing the program, result_of_conv1.txt, result_of_conv2.txt, result_of_fc.txt, and execution_result.txt will be generated. Please refer to the sample files provided on ee class. The sample content of execution_result.txt is as follows:

```
Relative Error after Quantization: 0.41%
Result of the Prediction: 5
Energy Consumption: 42110.0
Step Count: 2640
```

# 8. Bonus

The bonus part asks you to write a C++ program to generate your own test data. You can take a photo of a handwritten digit on white paper. The input format can be either **PGM ASCII (P2)** or **grayscale PNG**, we will provide both for you. You can choose which format to use as input for your program. Your program should perform resizing, invert colors (black background, white digit), FP8 conversion, and output the result to a text file. [7]



Figure 20 Samples from the Handwritten Digit Dataset

The program must complete the following preprocessing steps and save the output as img_label_bonus.txt:

1. Read the image: choose either PGM ASCII (P2): grayscale matrix or grayscale PNG whose pixel values is between 0–255.
2. Resize the image to 28×28 pixels.
3. Invert the colors to black background and white digit.

4. Convert the pixel values to FP8 format, mapped to the range of 0 to 1.
5. Write the result to img_label_bouns.txt: Each pixel separated by a comma, 28 pixels per line, and values rounded to ten decimal places, Following the same storage format as img_label.txt.

Afterwards, the bonus_img.txt output can be used as input for the basic part of the project to verify whether the model can correctly predict the handwritten digit. Testing will be performed using your own square handwritten digit images or converted PGM ASCII grayscale images (200×200 pixels). Example files will be provided on eeclass.

The following libraries are permitted. Other image processing or AI libraries are **prohibited**. You must implement resize and threshold functions yourself.

- <iostream>, <fstream>, <vector>, <array>, <cmath>, <iomanip>
- OpenCV (basic image processing functions only): allowed for reading images, grayscale conversion, resizing, thresholding, and color inversion. Only the following functions may be used: cv::imread(), cv::resize(), cv::cvtColor(), cv::threshold(), cv::bitwise_not()

**PGM Format Brief Explanation:**

```
P2
200 200
255
253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253
253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253
253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253
253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253 253
```

Figure 21 Samples Format of PGM File

Line 1: P2, means pixel values will present in ASCII grayscale image
Line 2: image width and height (width height)
Line 3: maximum pixel value (usually 255)
Following lines: pixel values arranged in row-major order

# 9. Submission Requirement

This project requires two people to work as a team. Each person should submit the following files. The team registration list will be announced later. If you only finish by yourself without a member, just key in one student ID.

1. Source code file (not the entire project) named:
   StudID1_StudID2_Final_Project.cpp
   (e.g., 114501000_114501500_Final_Project.cpp)
2. Report named:
   StudID1_StudID2_Final_Project_report.pdf
   (e.g., 114501000_114501500_Final_Project_report.pdf).
3. If you implement a bonus version, please submit it separately from the original

version. Name the source code file of the bonus:
StudID1_StudID2_Final_Project_bonus.cpp
(e.g., 14501000_114501500_Final_Project_bonus.cpp)

Note that the only acceptable report file format is .pdf, **no .doc/.docx or other files are acceptable. BE SURE to follow the naming rule mentioned above. Otherwise, your program will not be graded.**

We don't restrict the report format and length. In your report, you must at least describe:

- How to compile and execute your program; (You can use a screenshot to explain. In this final project, it can be done in one click)
- The degree of completion of the assignment; (If you complete all requirements, just specify all)
- The hardness of this assignment and how you overcome it.
- Any suggestions about this programming assignment?
- Briefly explain how you completed the Bonus Part if you did. Your workflow or only a simple description of your logic are all accepted.

You can also put anything related to the Final Project in your report, such as pseudocode, control flow diagram, programming developing thought, etc. Your program will be judged by checker written by TA 彭芮誼. Appendix 1 will explain how to add header to your project. The machine information is asked as follows:

- OS: Ubuntu 20.04 LTS
- OJ Version：Hydro OJ v2.33.13
- Compiler：GNU GCC 9.3.0
- Supporting C++ version：C++ 11

**One important point to note is that we will only run each case for 1 minute. If there is no output within that minute, the case will receive a score of 0.**

# 10. Grading & Criteria

The grading is as follows:

- Correctness of your code (50%): Able to tell the relative error after quantization and the result of the prediction. We will apply checker on the output file your code generate. There will be hidden test cases.
- Readability of your code (10%)
- The report (20%)
- Optimization (20%):
  This section evaluates the step count and energy based on follow criteria:
  1. Comparison with the baseline result for both energy and step(12%):
  - If the result **exceeds** the baseline by 25%, a score of 0 is given.

- If the result **exceeds** the baseline by 0~25%, a score of 1 is given.
- If the result **lowers** the baseline by 0~25%, a score of 1.5 is given.
- If the result **lowers** the baseline by more than 25%, a score of 2 is given.

The baseline for different kernel size is shown as follow, two points for each index:

| Kernel | Energy | Step |
|---|---|---|
| 3*3 | 192,290 | 6,408 |
| 5*5 | 697,160 | 23,232 |
| 7*7 | 946,380 | 31,536 |

2. Comparison with the class performance(8%):

An additional 8 points will be distributed based on the ranking among all students according to their step count and energy consumption.

- Bonus (10%)

NO demo session will be held for final project!!!!

NO LATE SUBMISSION is accepted!!!!! Please submit your assignment on time.

If you have questions, please E-mail to both me (andyygchen@ee.ncu.edu.tw) and TA 彭芮誼 (c111501541@g.ncu.edu.tw).

# 11. Reference

[1] Chen, Y.-H., Emer, J., & Sze, V. (2017). Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. IEEE Journal of Solid-State Circuits, 52(1), 127–138.

[2] Juracy, L. R., Garibotti, R., & Moraes, F. G. (2023). From CNN to DNN Hardware Accelerators: A Survey on Design, Exploration, Simulation, and Frameworks. Foundations and Trends® in Electronic Design Automation, 13(4), 270-344.

[3] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, R. Ho, M. Jones, N. Jouppi, J. Kurzak, N. Luehr, S. Markidis, M. Mudigonda, J. Park, S. Pennycook, M. Siu, and C. Sun, “FP8 formats for deep learning,” *arXiv preprint* arXiv:2209.05433, 2022.

[4] Tse, J. (2024, December 9). *Understanding the FP64, FP32, FP16, BFLOAT16, TF32, and FP8 formats*. Jeffrey Tse’s Blog. https://jeffreytse.net/computer/2024/12/09/understanding-the-fp64-fp32-fp16-bfloat16-tf32-fp8-formats.html

[5] Machine Learning 2021: Convolutional Neural Networks (CNN)” [Video]. YouTube. https://www.youtube.com/watch?v=OP5HcXJg2Aw

[6] Jouppi, N. P., Young, C., Patil, N., et al. (2017). In-datacenter performance analysis of a tensor processing unit. Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA), 1–12.

[7] PGM File Format on Wikipedia https://en.wikipedia.org/wiki/Netpbm#PGM_example